SEHAT SETU
Patna Hospital Information & Cost Transparency Portal

Capstone Project Proposal

Institute: IIT Patna

Instructor: Dr. Kuldeep Singh Patel

Team: Himanshu (Ramnagar, West Champaran), Kaushlendra (Rajgir), Keshav (Lakhisarai), Himanshu (Sheikhpura), Jitesh (East Champaran)

Duration: 3 months

Tagline: Sehat ka Setu, Har Ghar Tak

SehatSetu

Sehat Ka Setu, Har Ghar Tak

Date: 29 January 2026

Acknowledgement

We express our sincere gratitude to our instructor, Dr. Kuldeep Singh Patel, for his guidance, and to our institute for enabling this mandatory capstone. We also thank hospital administrators in Patna who provided publicly available information and clarifications during data verification calls. Finally, we appreciate the encouragement of our families and peers.

Abstract

Sehat Setu is a city- focused hospital information portal that aggregates verified details of hospitals in Patna, including facilities, doctor rosters, indicative treatment cost ranges, and insurance tie- ups. The project aims to improve transparency, reduce search frictions for citizens, and provide a mobile- friendly interface for quick comparisons. The system is implemented using Python (Flask), SQLite for data storage, and Bootstrap for the user interface. We outline requirements, architecture, data model, collection workflows, testing, and ethical safeguards, and we present an evaluation with a seeded dataset of Patna hospitals.

Table of Contents

1. Introduction

2. Problem Statement & Objectives

3. Scope & Use Cases

4. Literature Survey & Background

5. System Requirements

6. Project Management (Team, Timeline, Risks)

7. System Design (Architecture & Data Model)

8. UI/UX Design

9. Implementation (Backend, Frontend, Database)

10. Data Collection & Curation

11. Search, Filters & Algorithms

12. Security, Privacy & Ethics

13. Accessibility, Localization & Inclusivity

14. Testing & Quality Assurance

15. Results & Evaluation

16. Deployment & Operations

17. Cost, Sustainability & Future Work

18. Conclusion

19. References (to be updated)

Appendix A: ER Diagram (description)

Appendix B: API Endpoints

Appendix C: SQL Schema

Appendix D: Test Cases

Appendix E: Data Collection Templates

Appendix F: Consent & Communication Templates

1. Introduction

Healthcare decisions are time- sensitive and emotionally taxing. In many Indian cities, citizens rely on word of mouth or fragmented web results to locate hospitals, understand available facilities, and estimate likely costs. While some chains publish package pricing, independent hospitals vary widely in how information is presented. This leads to information asymmetry, surprise billing, and sub- optimal hospital selection.

Sehat Setu attempts to bridge this gap for Patna by building a curated, structured, and verifiable directory that surfaces: (a) hospital profile and facilities, (b) doctor roster and timings, (c) treatment cost bands, and (d) insurance tie- ups including cashless availability. The initial focus is Patna city, with a plan to expand to nearby districts in later phases.

The system is built as a web application with a lightweight architecture suitable for a student capstone: Flask for routing and server logic, SQLite for persistence, and a Bootstrap- based responsive UI. The project emphasizes data ethics, transparency, and maintainability.

2. Problem Statement & Objectives

Problem Statement: Lack of a single, trustworthy, citizen- friendly portal that aggregates Patna hospital information with transparent cost ranges and insurance coverage results in time loss, anxiety, and possible financial mis- planning for patients and families.

Primary Objectives: (1) Build a hospital directory for Patna with verifiable entries; (2) Publish indicative treatment cost ranges along with assumptions; (3) Show insurance partners and cashless availability; (4) Provide search, filter, and comparison to assist decision- making; (5) Enable data stewardship workflows for periodic verification and updates.

Secondary Objectives: (a) Multilingual support (English/Hindi), (b) Offline- first caching for low bandwidth users, (c) Export of selected hospital lists to PDF for caregivers, (d) Open data mindset with CSV export for research (subject to consent).

3. Scope & Use Cases

In-scope: Public information about hospitals in Patna, doctor specialties, facilities, address/contact, indicative costs for common treatments (e.g., normal delivery, C-section, appendectomy, cataract), insurance panels, and seasonal health check packages or discounts.

Out-of-scope: Personalized medical advice, emergency triage, real-time bed availability, and handling of sensitive personal health records. The portal is informational, not a replacement for clinical consultation.

Key Use Cases: (UC-1) Citizen searches for hospitals offering cardiology with ICU and cashless insurance; (UC-2) Pregnant mother compares delivery cost packages across 3 hospitals; (UC-3) Senior citizen filters for hospitals within a specific budget range; (UC-4) Researcher exports anonymized summary statistics; (UC-5) Admin updates hospital entry after a quarterly verification call.

4. Literature Survey & Background

Hospital information systems and consumer- facing portals typically focus on one aspect: appointments, pharmacy, diagnostics, or telemedicine. City- level cost transparency is less common, especially for smaller independent hospitals. Internationally, several jurisdictions encourage publishing ranges rather than exact prices to account for variability by room type, consumables, and complications.

Our design borrows proven ideas: structured provider directories, schema- based data collection, and progressive disclosure of details. We maintain clear disclaimers and encourage direct confirmation with hospital billing desks for final estimates.

5. System Requirements

Functional Requirements: (FR- 1) Hospital CRUD for admins; (FR- 2) Public hospital listing and detail pages; (FR- 3) Search across name, area, specialty, treatment; (FR- 4) Filters for facility, insurance, cost band, hospital type; (FR- 5) Comparison of up to three hospitals; (FR- 6) Multilingual labels; (FR- 7) CSV export for selected records; (FR- 8) Admin login with role- based permissions.

Non- Functional Requirements: (NFR- 1) Page load under 3 seconds on local demo; (NFR- 2) Mobile- first responsiveness; (NFR- 3) Data integrity with foreign keys; (NFR- 4) Secure password hashing for admin accounts; (NFR- 5) Audit trail for edits; (NFR- 6) Accessibility with keyboard navigation and alt text; (NFR- 7) Graceful degradation on slow networks.

6. Project Management (Team, Timeline, Risks)

Team Roles: Kaushalendra — Data lead & hospital verification; Keshav — Backend (Flask, APIs); Himanshu (Sheikhpura) — Frontend (Bootstrap, UI flows); Himanshu (Ramnagar) — Data import & treatments/cost model; Jitesh — Maps & documentation.

Timeline: Month- 1 foundation (UI wireframes, DB schema, Flask skeleton); Month- 2 implementation (pages, search, filters, data seeding); Month- 3 polish (compare view, QA, report/PPT).

Risks & Mitigation: (R- 1) Incomplete data → prioritize top 50 hospitals first; (R- 2) Inconsistent cost formats → enforce a min- max numeric schema; (R- 3) Scope creep → MVP freeze after Week- 4; (R- 4) Team bandwidth → weekly stand- ups and Kanban in a shared board.

7. System Design (Architecture & Data Model)

Architecture: A three- layered stack — (a) Presentation (HTML/CSS/Bootstrap), (b) Application (Flask routes, templates, controllers), and (c) Data (SQLite). Static assets served by Flask; ORM via lightweight SQLite queries with parameter binding. Caching can be added via simple in- memory maps for frequent lists.

Data Model: Core tables — hospitals, doctors, treatments, insurance, discounts, and optional reviews. Foreign keys maintain referential integrity. Facilities stored as JSON to allow flexible extensions (e.g., adding MRI/NICU/ambulance flags without schema break).

8. UI/UX Design

Design Principles: clarity, contrast, consistency, and compassion. Use high- contrast fonts, clear call- to- action buttons, and minimal forms. The home page features a universal search bar and quick chips for specialties and facilities.

Information Architecture: Home → Hospitals (list) → Hospital Detail → Compare; parallel tabs for Doctors, Treatments, and Insurance. Sticky footer includes disclaimers and contact email for corrections.

9. Implementation (Backend, Frontend, Database)

Backend: Flask app with blueprint modules — public (browse, search), admin (CRUD), services (csv import/export). Jinja2 templates for server- side rendering; minimal JS for interactivity. Passwords hashed with a standard algorithm (e.g., PBKDF2 in Werkzeug).

Frontend: Bootstrap grid, form controls for filters, badges for facilities, and cards for hospital summaries. Client- side enhancements use progressive enhancement to keep pages usable without JS.

Database: SQLite schema with indices on (name, area) and composite indices for search filters (hospital_id + treatment_name). All writes validated server- side with value ranges (e.g., cost_min <= cost_max).

10. Data Collection & Curation

Workflow: Prepare Google Sheets templates, collect public information from websites/phone calls, normalize into CSV, validate types, and import. Each record includes last_verified_on date and a source_note field for traceability.

Quality Controls: duplicate detection by fuzzy matching on name + address, mandatory phone number pattern, and threshold checks for outlier costs. Admins can lock verified records for a quarter.

11. Search, Filters & Algorithms

Search: Simple LIKE queries with tokenization; future improvement — trigram index for fuzzy search. Filters combine via AND logic with indexed columns. Sorting supports cost_low_to_high (uses treatment median) and distance (if geolocation enabled).

Comparison: Normalize features into a comparable grid (cost band, facilities flags, insurance presence). Calculate a simple suitability score (weighted sum based on user- selected priorities).

12. Security, Privacy & Ethics

No personal health data stored in MVP. Admin logins protected with hashed passwords and session timeouts. Input sanitized and SQL parameters used to prevent injection. HTTPS recommended in deployment.

Ethics: Present costs as indicative ranges with disclaimers; encourage users to confirm with hospital billing. Provide a transparent correction channel for hospitals to request updates.

13. Accessibility, Localization & Inclusivity

Accessibility: Semantic HTML, keyboard navigability, focus indicators, alt text for icons, sufficient color contrast, and form labels associated with inputs.

Localization: Bilingual labels (English/Hindi) and support for Devanagari font rendering; date/currency formatting uses Indian conventions (₹, lakh format).

14. Testing & Quality Assurance

Test Plan: Unit tests for data validators; integration tests for key routes; manual UI tests across desktop and mobile. Data verification tests ensure cost ranges are in realistic bounds.

Sample KPIs: Page render time (<3s local), correctness of top- 5 search results, and 0 critical accessibility violations in a basic audit.

15. Results & Evaluation

With a seeded dataset of representative Patna hospitals, Sehat Setu demonstrates fast search and clear comparison cards. Pilot feedback from students and families suggests improved understanding of cost ranges and insurance availability.

Limitations: manual data updates require sustained effort; exact prices vary by case; and some independent clinics do not publish enough information publicly.

16. Deployment & Operations

Deployment Options: local demo on Flask built- in server; staging on a free cloud host; future move to a managed platform. Static asset minification and image compression recommended.

Operations: quarterly verification cycle, backup of SQLite DB, and change logs for edited entries.

17. Cost, Sustainability & Future Work

Cost: student- friendly stack keeps costs near zero for development and testing. Optional expenses include domain name and hosting.

Future Work: add user reviews with moderation, geolocation and distance sorting, CSV open data portal, and an admin dashboard with bulk import and validation rules.

18. Conclusion

Sehat Setu demonstrates that a focused, ethical, and data- centric approach can make hospital information more accessible for citizens. The project balances technical feasibility with social impact and establishes a repeatable blueprint for other cities.

19. References (to be updated)

[1] Hospital websites and brochures (public information).

[2] Telephone/email clarifications from hospital desks (where obtained).

[3] General web design and accessibility best practices.

Appendix A: ER Diagram (description)

Entities: HOSPITAL (id, name, type, address, contact, facilities_json, rating_avg, last_verified_on), DOCTOR (id, hospital_id, name, specialization, experience_years, op_timings, days_available), TREATMENT (id, hospital_id, treatment_name, cost_min, cost_max, notes), INSURANCE (id, hospital_id, insurer_name, cashless_available, tpa_details), DISCOUNT (id, hospital_id, title, description, discount_pct, valid_from, valid_till).

Relationships: HOSPITAL 1- to- many with DOCTOR, TREATMENT, INSURANCE, DISCOUNT.

Appendix B: API Endpoints

GET / — Home page with search

GET /hospitals — List hospitals

GET /hospital/<id> — Hospital detail

GET /doctors — Doctors list (optional)

GET /search?q= — Combined search

POST /admin/hospital — Create (admin)

PUT /admin/hospital/<id> — Update (admin)

DELETE /admin/hospital/<id> — Delete (admin)

Appendix C: SQL Schema

```
CREATE TABLE hospitals (id INTEGER PRIMARY KEY, name TEXT NOT NULL, type TEXT, address TEXT, area TEXT, city TEXT, pincode TEXT, latitude REAL, longitude REAL, contact_phone TEXT, contact_email TEXT, website TEXT, facilities_json TEXT, rating_avg REAL, last_verified_on TEXT);

CREATE TABLE doctors (id INTEGER PRIMARY KEY, hospital_id INTEGER, name TEXT, specialization TEXT, experience_years INTEGER, qualifications TEXT, op_timings TEXT, days_available TEXT, contact_ext TEXT, FOREIGN KEY(hospital_id) REFERENCES hospitals(id));

CREATE TABLE treatments (id INTEGER PRIMARY KEY, hospital_id INTEGER, treatment_name TEXT, cost_min REAL, cost_max REAL, notes TEXT, FOREIGN KEY(hospital_id) REFERENCES hospitals(id));

CREATE TABLE insurance (id INTEGER PRIMARY KEY, hospital_id INTEGER, insurer_name TEXT, cashless_available TEXT, tpa_details TEXT, FOREIGN KEY(hospital_id) REFERENCES hospitals(id));

CREATE TABLE discounts (id INTEGER PRIMARY KEY, hospital_id INTEGER, title TEXT, description TEXT, discount_pct REAL, valid_from TEXT, valid_till TEXT, terms TEXT, FOREIGN KEY(hospital_id) REFERENCES hospitals(id));
```

Appendix D: Test Cases

TC- 01 Search by hospital name returns correct record.

TC- 02 Filter by facility ICU returns only hospitals with ICU flag true.

TC- 03 Cost band filter (<= 50,000) returns matching treatments.

TC- 04 Admin cannot set cost_min > cost_max (validation error).

TC- 05 CSV import rejects duplicate hospitals (same name + address).

TC- 06 Pages meet minimum contrast ratio and include alt text.

Appendix E: Data Collection Templates

Hospital Master: name, type, address, area, phone, website, facilities, emergency (Y/N).

Cost Sheet: hospital name, treatment, cost_min, cost_max, notes.

Insurance Sheet: hospital name, insurer, cashless (Y/N), TPA.

Appendix F: Consent & Communication Templates

Email to hospital: Request to confirm public information, including facilities and insurance panels, for citizen awareness.

Phone script: Introduce project purpose, verify key fields, and note date/time of verification.